

Understanding graph representation learning in RL-based logic synthesis

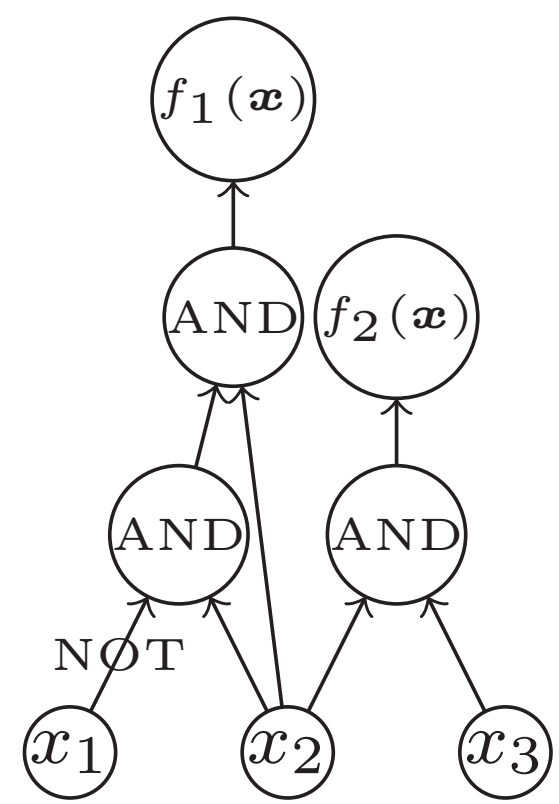
Hector Kohler, Johannes Lutzeyer, Jesse Read

LIX, Ecole Polytechnique, CNRS, Institut Polytechnique de Paris

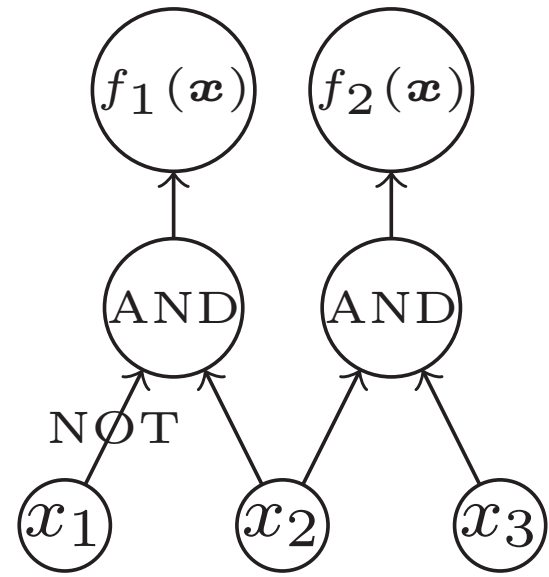


1. Logic synthesis with AIGs

- Chip design needs the smallest circuit encoding a given Boolean function (NP).
- In practice means the smallest and-inverter graph (AIG): nodes are *AND* gates and edge labels are *NOT* signals.



$\mathcal{G}_1$: 8 nodes, 4 levels.

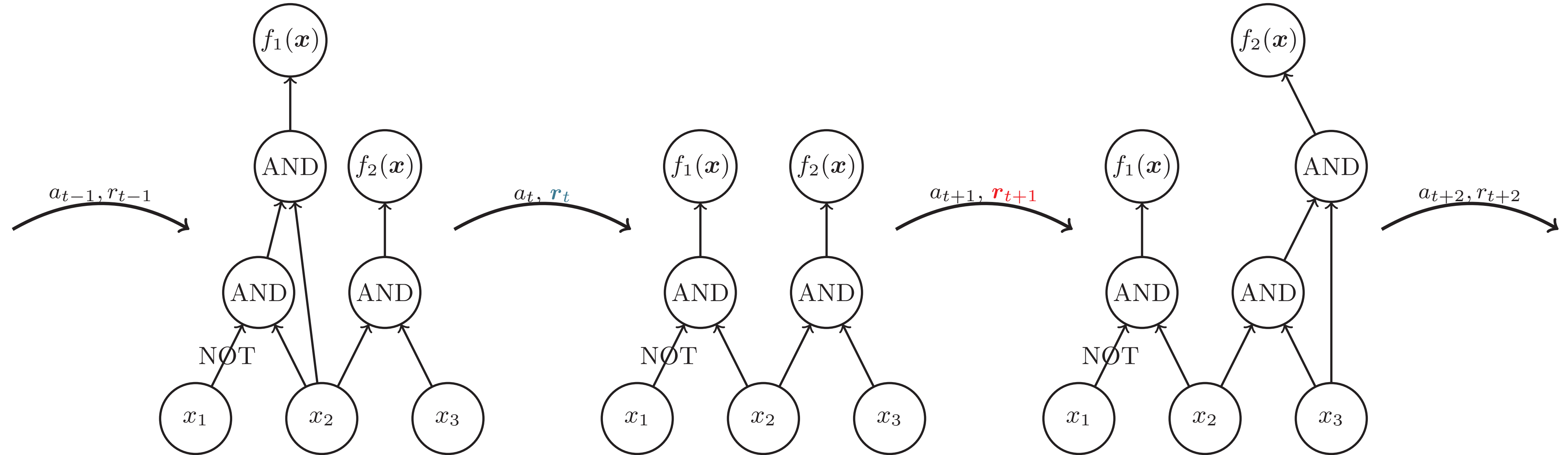


$\mathcal{G}_2$: 7 nodes, 3 levels.

Two functionally equivalent AIGs of $f(x_1, x_2, x_3) = (\overline{x_1} \wedge x_2, x_2 \wedge x_3)$ with different numbers of nodes.

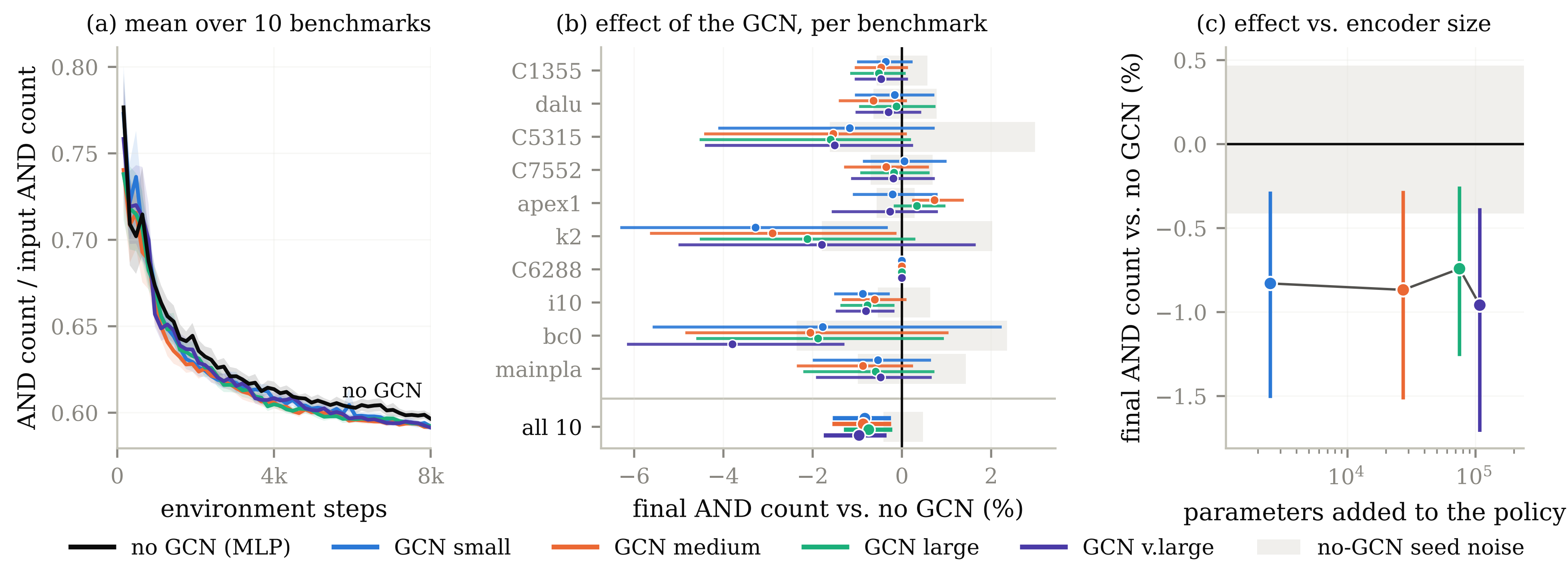
2. AIG reduction as a Markov decision process

- Each action is one ABCprimitive applied to the whole graph [3] and the reward grows as the number of *AND* nodes drops; primitives preserve functional equivalence.
- Most RL and other machine learning-based searches act at this granularity [6, 2, 4].
- DAG generative models fail to generate functionally equivalent AIGs [5].



3. Graph representation and RL

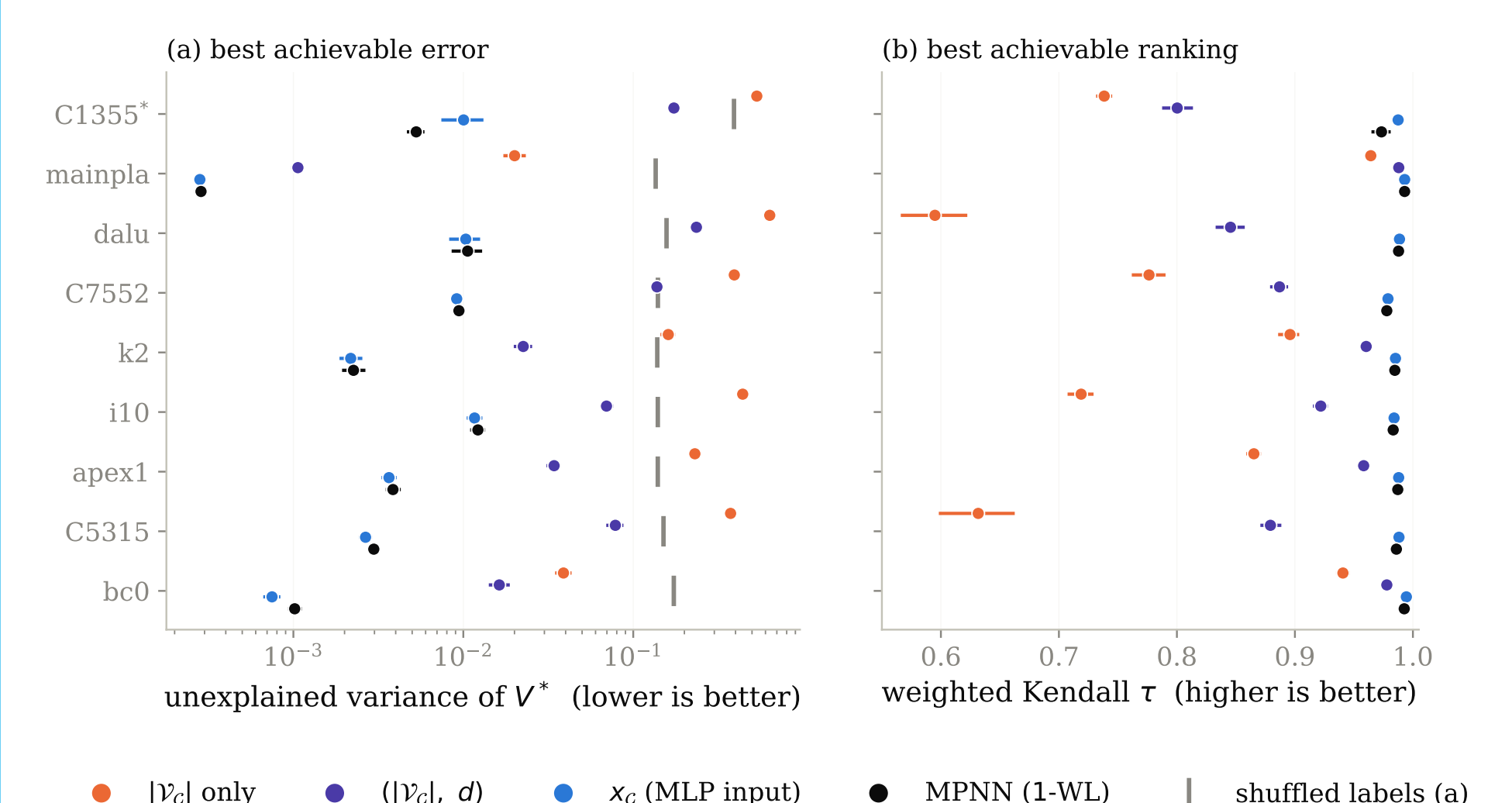
We train **tuned** PPO agents on the ten benchmarks of Zhu et al. [6] and compare GCN policies against policies that only see graph statistics (current # nodes, # edges, last few graph edits taken, current time step).



4. Final AIG sizes

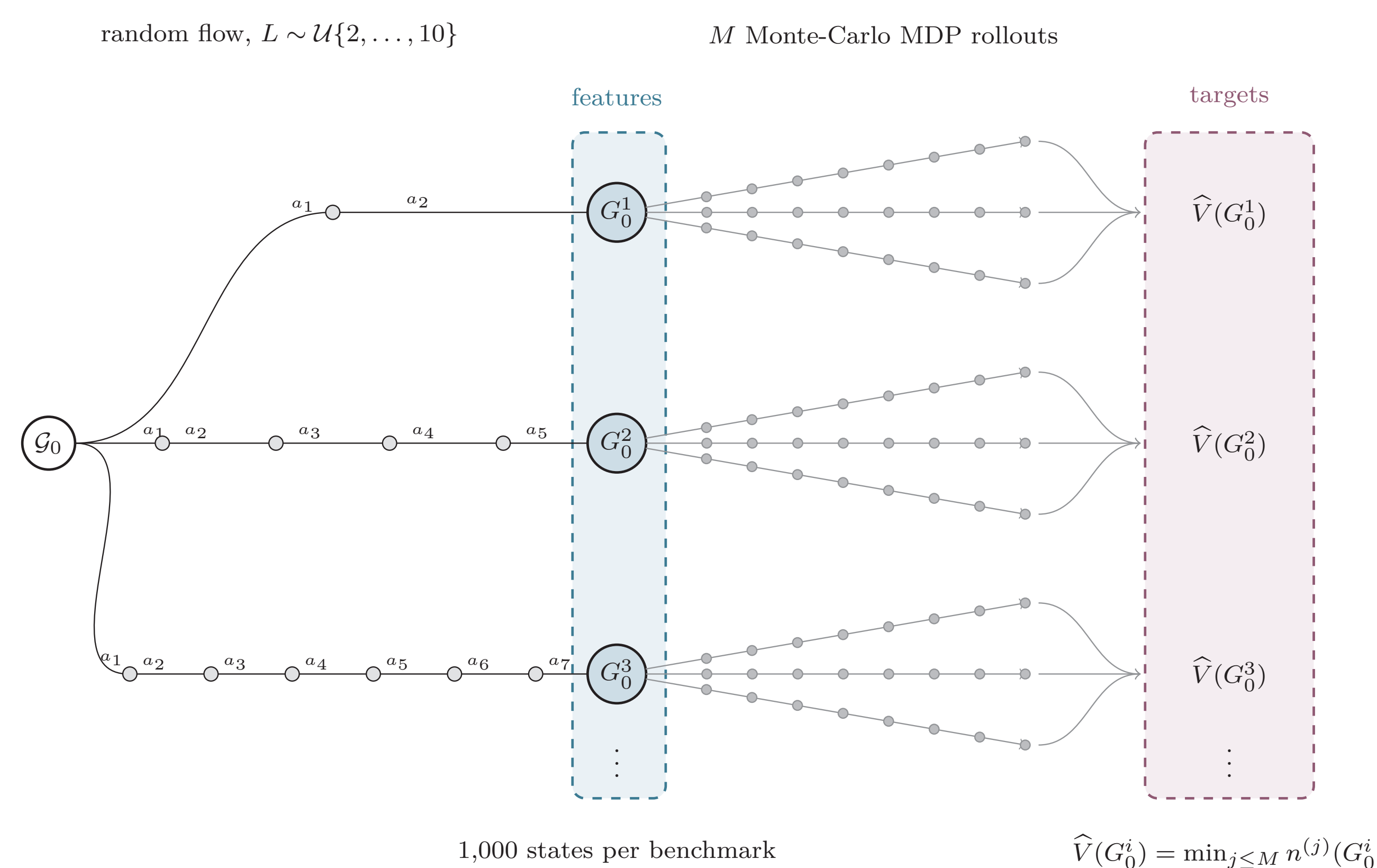
AIG	Starting nodes	no GCN (MLP)	GCN small	GCN medium	GCN large	GCN v.large	resyn2
C1355	504	389 ± 3	387 ± 1	387 ± 0	387 ± 1	387 ± 0	390
dal	1740	1008 ± 9	1006 ± 6	1001 ± 3	1006 ± 6	1005 ± 2	1052
C5315	1773	1303 ± 43	1287 ± 8	1283 ± 3	1282 ± 6	1283 ± 5	1309
C7552	2074	1371 ± 12	1372 ± 10	1367 ± 11	1369 ± 5	1369 ± 10	1455
apex1	2124	1066 ± 7	1064 ± 11	1074 ± 4	1070 ± 4	1063 ± 14	1179
k2	2190	1049 ± 26	1015 ± 32	1019 ± 27	1027 ± 21	1030 ± 36	1194
C6288	2337	1870 ± 0	1870 ± 0	1870 ± 0	1870 ± 0	1870 ± 0	1870
i10	2675	1715 ± 13	1700 ± 3	1705 ± 9	1702 ± 3	1702 ± 6	1829
bc0	3501	1512 ± 46	1485 ± 59	1481 ± 31	1484 ± 28	1455 ± 13	1713
mainpla	6359	2318 ± 37	2306 ± 14	2298 ± 13	2305 ± 23	2307 ± 14	2476

6. 1-WL representation error bounds

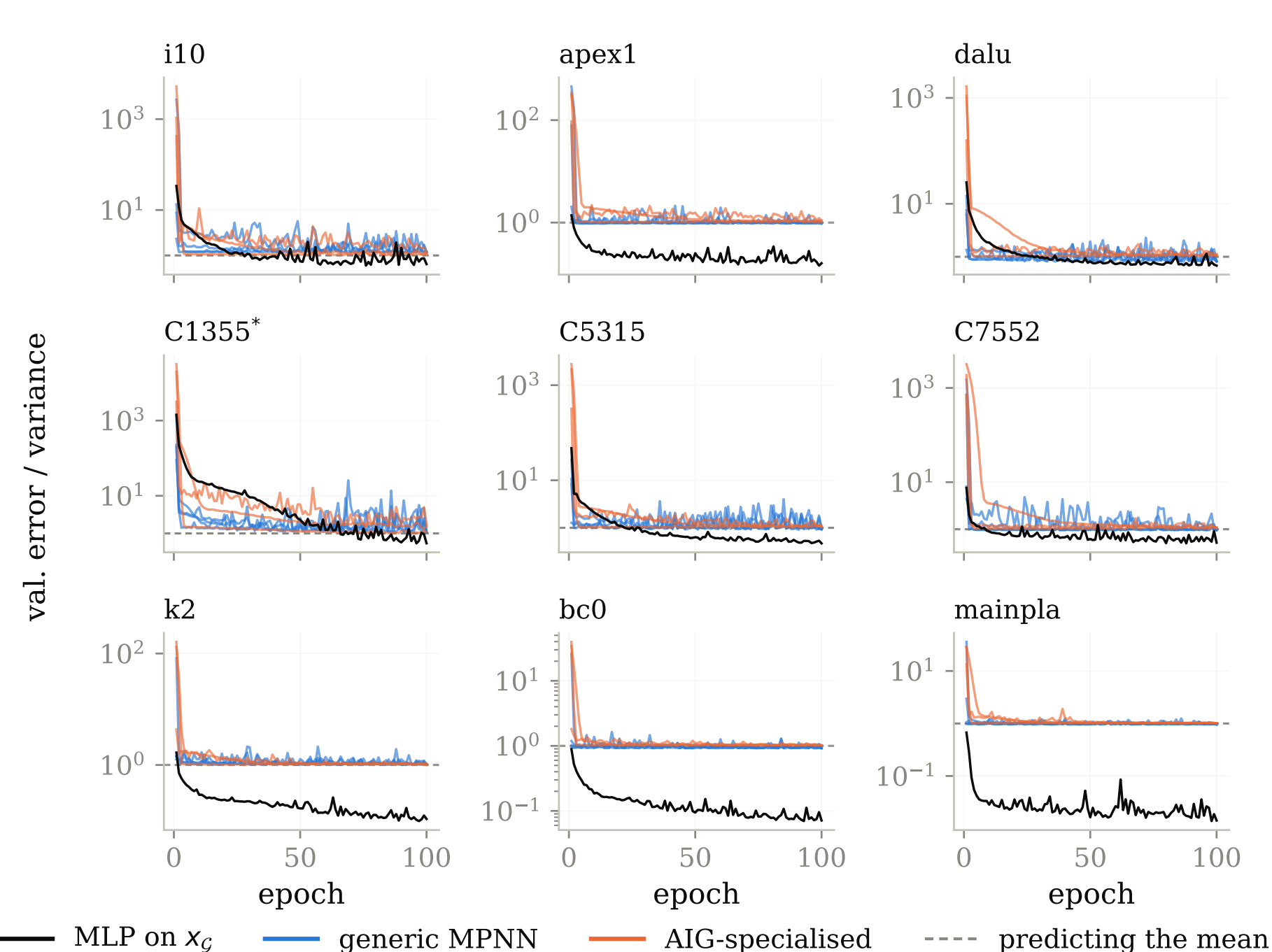


5. Why don't graph representation help?

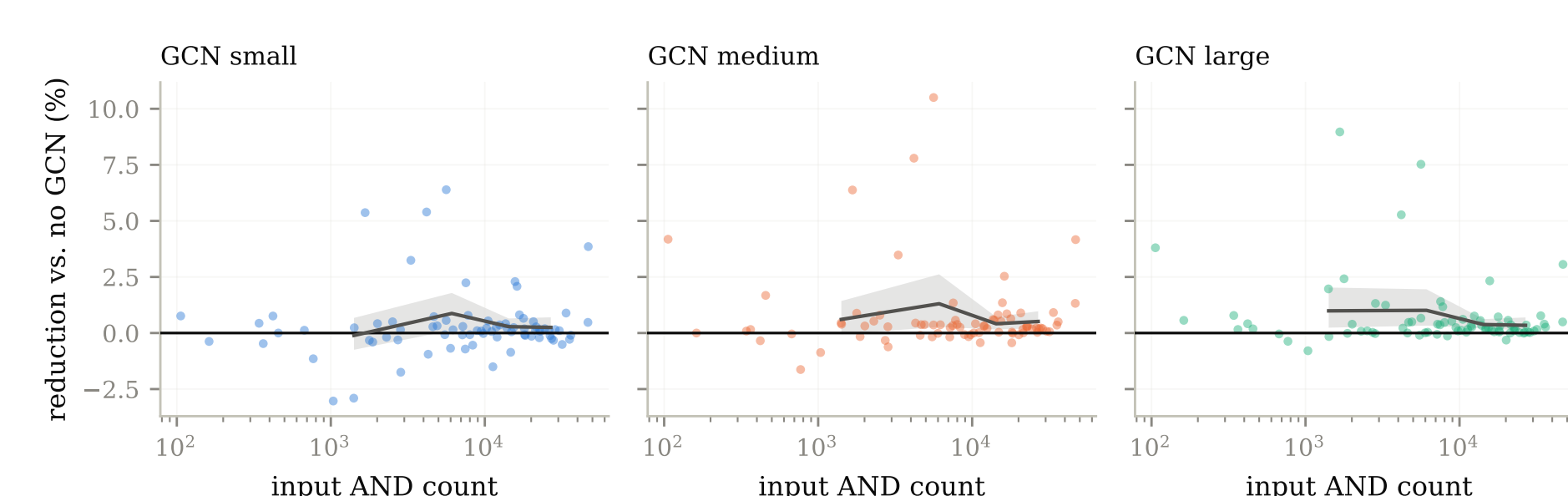
- Computing V^* exactly is intractable, so we estimate it.
- With this supervised dataset we can study the predictive power of message passing networks (1-WL) on AIGs with Graphtester [1].



7. AIG-specific value models



Additional results and future work



- Extend the analysis to non 1-WL representation, since some AIG-specific models might not be 1-WL.
- ABC implements much more than 5 primitives, but no search method uses them all. More actions would mean more varied states, which could be where graph representation learning helps.

Key references

- [1] E. Akbiyik, F. Grötschla, B. Egressy, and R. Wattenhofer. Graphtester: Exploring Theoretical Boundaries of GNNs on Graph Datasets. In *Data-centric Machine Learning Research (DMLR) Workshop at ICLR 2023, Honolulu, Hawaii*, July 2023.
- [2] A. Hosny, S. Hashemi, M. Shalan, and S. Reda. Drills: Deep reinforcement learning for logic synthesis. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 581–586, 2020.
- [3] A. Mishchenko, S. Chatterjee, and R. Brayton. Dag-aware aig rewriting: a fresh look at combinational logic synthesis. In *2006 43rd ACM/IEEE Design Automation Conference*, pages 532–535, 2006.
- [4] W. L. Neto, Y. Li, P.-E. Gaillardon, and C. Yu. Flowtune: End-to-end automatic logic optimization exploration via domain-specific multiarmed bandit. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(6):1912–1925, 2023.
- [5] T. Stoll, C. Qian, B. Finkelshtein, A. Parviz, D. Weber, F. Frasca, H. Shavit, A. Siraudin, A. Mielke, M. Anastacio, E. Müller, M. Bechler-Speicher, M. Bronstein, M. Galkin, H. Hoos, M. Niepert, B. Perozzi, J. Tönschoff, and C. Morris. Graphbench: Next-generation graph learning benchmarking, 2026. URL <https://arxiv.org/abs/2512.04475>.
- [6] K. Zhu, M. Liu, H. Chen, Z. Zhao, and D. Z. Pan. Exploring logic optimizations with reinforcement learning and graph convolutional network. In *2020 ACM/IEEE 2nd Workshop on Machine Learning for CAD (MLCAD)*, pages 145–150, 2020.